

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Направление подготовки **09.03.03 Прикладная информатика**

Образовательная программа **Мобильные и сетевые технологии**

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
по дисциплине “Проектирование и реализация баз данных”

Обучающийся: Захаркин Богдан Владимирович, К3440

Преподаватель: Говорова Марина Михайловна

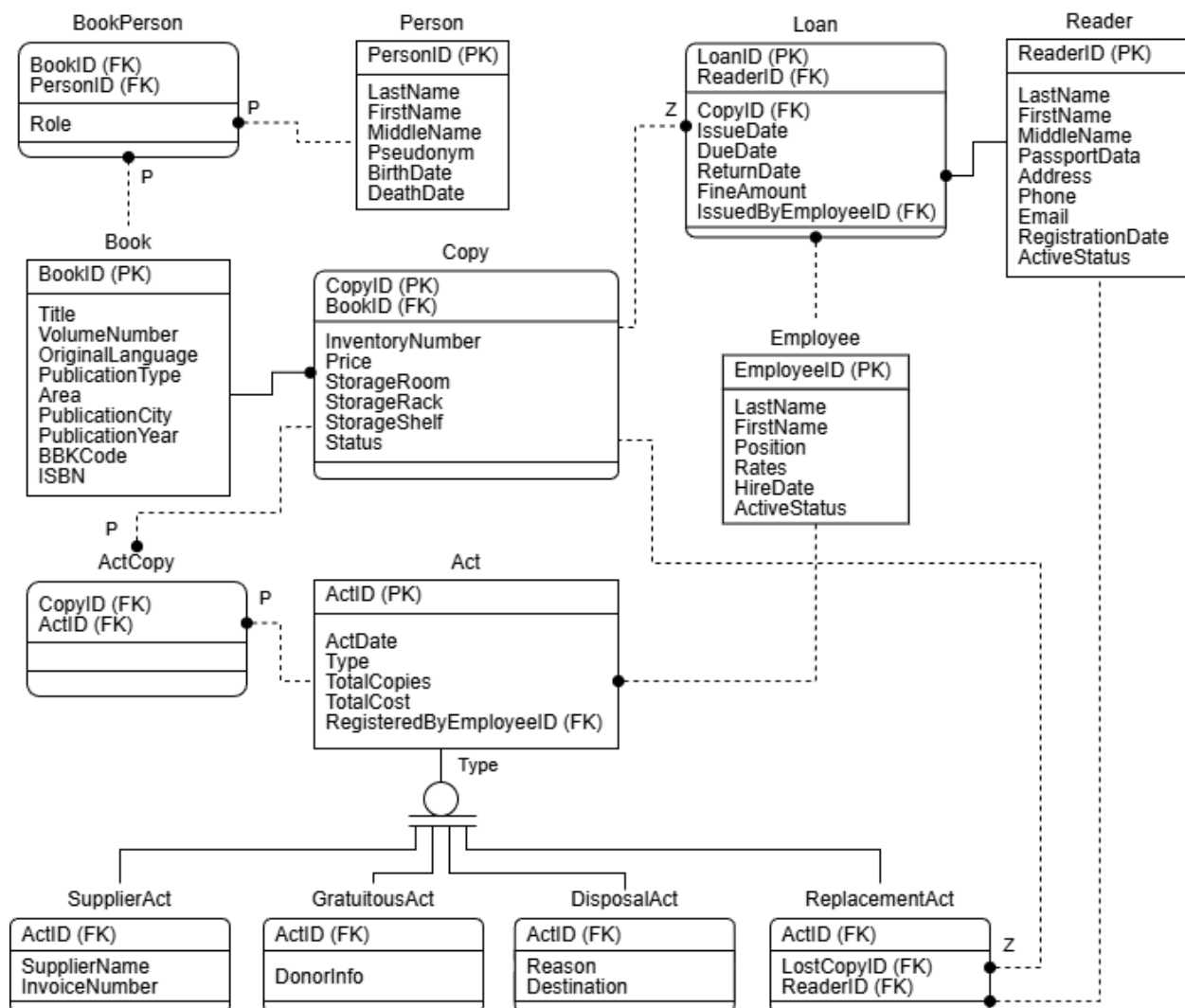
Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

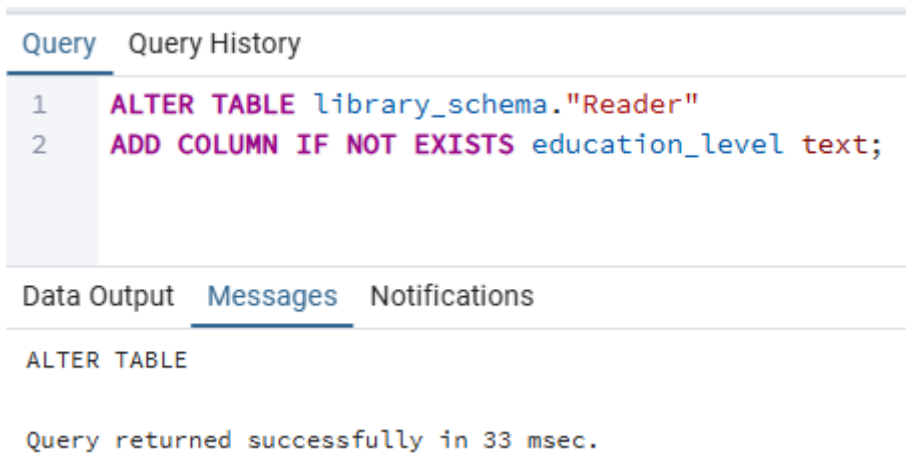
Выполнение:

Схема library:



1 Подготовка

Добавил столбец `education_level`, так как в одном из запросов понадобится уровень образования. В диаграмму я его не добавил, так как я его не планировал добавлять, так будет и дальше.



The screenshot shows a SQL query editor interface. At the top, there are two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying a two-line SQL command: `1 ALTER TABLE library_schema."Reader"` and `2 ADD COLUMN IF NOT EXISTS education_level text;`. Below the query editor, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the text 'ALTER TABLE' and a confirmation message: 'Query returned successfully in 33 msec.'

```
Query Query History
1 ALTER TABLE library_schema."Reader"
2 ADD COLUMN IF NOT EXISTS education_level text;

Data Output Messages Notifications
ALTER TABLE

Query returned successfully in 33 msec.
```

К сожалению остальных скринов нет, но там я присвоил всем существующим читателям среднее образование и немного дополнил таблицы, например книгой про C#.

2 Создание запросов

– Вывести список читателей, имеющих на руках книги, переведенные с английского языка, изданные позднее 2000 года.

Query Query History

```
1 SELECT DISTINCT
2     r.reader_id,
3     r.lats_name AS last_name,
4     r.first_name,
5     r.middle_name
6 FROM library_schema."Reader" r
7 JOIN library_schema."Loan" l ON l.reader_id = r.reader_id
8 JOIN library_schema."Copy" c ON c.copy_id = l.copy_id
9 JOIN library_schema."Book" b ON b.book_id = c.book_id
10 WHERE l.return_date IS NULL
11     AND b.original_language = 'eng'
12     AND EXTRACT(YEAR FROM b.publication_year) > 2000
13 ORDER BY r.lats_name, r.first_name;
```

Data Output Messages Notifications

	reader_id [PK] integer	last_name text	first_name text	middle_name text
1	1	Соколов	Алексей	Викторович

– Вывести список читателей, не вернувших в срок книги и имеющих на руках более десяти книг.

Я вывел читателей, которые имеют хоть сколько-то книг на руках, так как и таких мало (>0).

Query Query History

```
1 SELECT
2     r.reader_id,
3     r.lats_name,
4     r.first_name,
5     COUNT(*) AS books_on_hand
6 FROM library_schema."Reader" r
7 JOIN library_schema."Loan" l ON l.reader_id = r.reader_id
8 WHERE l.return_date IS NULL
9     AND l.due_date < CURRENT_DATE
10 GROUP BY r.reader_id, r.lats_name, r.first_name
11 HAVING COUNT(*) > 0
12 ORDER BY books_on_hand DESC;
```

Data Output Messages Notifications

	reader_id [PK] integer	last_name text	first_name text	books_on_hand bigint
1	2	Морозова	Ольга	2
2	3	Васильев	Павел	1

- Вывести список изданий, которые были наиболее популярными у читателей за предыдущий календарный год.

Я вывел топ-3, так как начиная с 3 начинаются книги, которые брали всего один раз.

Query

Query History

```

1  WITH loans_2025 AS (
2      SELECT
3          b.book_id,
4          b.title,
5          COUNT(*) AS loan_count
6      FROM library_schema."Loan" l
7      JOIN library_schema."Copy" c ON c.copy_id = l.copy_id
8      JOIN library_schema."Book" b ON b.book_id = c.book_id
9      WHERE l.issue_date >= '2025-01-01'
10         AND l.issue_date < '2026-01-01'
11      GROUP BY b.book_id, b.title
12 )
13 SELECT |
14     title,
15     loan_count AS times_issued_in_2025
16 FROM loans_2025
17 ORDER BY loan_count DESC
18 LIMIT 3;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🔍

⬇️

⚡

SQL

	title text	times_issued_in_2025 bigint
1	Преступление и наказание	2
2	Гарри Поттер и философский каме...	2
3	Война и мир	1

– Вывести список книг, которые находятся в библиотеке в единственном экземпляре.

Query

Query History

```

1  SELECT
2      b.book_id,
3      b.title,
4      b.publication_year::date AS pub_year,
5      COUNT(c.copy_id) AS copies_count
6  FROM library_schema."Book" b
7  LEFT JOIN library_schema."Copy" c ON c.book_id = b.book_id
8  GROUP BY b.book_id, b.title, b.publication_year
9  HAVING COUNT(c.copy_id) = 1
10 ORDER BY b.title;

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

	book_id [PK] integer	title text	pub_year date	copies_count bigint
1	6	Harry Potter and the Chamber of Secr...	2002-01-01	1
2	10	Programming in C# Basics	2023-01-01	1
3	7	Test Book 1	2020-01-01	1
4	8	Test Book 2	2020-01-01	1
5	9	Unique Book	2010-01-01	1
6	5	Ведьмак. Последнее желание	2023-11-05	1
7	2	Война и мир	2020-05-15	1
8	3	Мастер и Маргарита	2021-03-10	1

- Подсчитать количество читателей, которые не обращались в библиотеку в течение года.

Query Query History

```

1 SELECT COUNT(DISTINCT r.reader_id) AS inactive_more_than_year
2 FROM library_schema."Reader" r
3 WHERE NOT EXISTS (
4     SELECT 1
5     FROM library_schema."Loan" l
6     WHERE l.reader_id = r.reader_id
7           AND l.issue_date >= '2025-01-18'::date - INTERVAL '1 year'
8 );

```

Data Output Messages Notifications

	inactive_more_than_year bigint
1	5

- Подсчитать количество читателей библиотеки по уровню образования.

Query Query History

```

1 SELECT
2     education_level,
3     COUNT(*) AS readers_count
4 FROM library_schema."Reader"
5 GROUP BY education_level
6 ORDER BY readers_count DESC;

```

Data Output Messages Notifications

	education_level text	readers_count bigint
1	среднее	7
2	высшее	2
3	среднее специальн...	1

– Вывести список книг по программированию на C#, экземпляры которых отсутствуют в библиотеке, и которые должны быть возвращены не позднее, чем через 3 дня.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

SELECT

b.title,

COUNT(l.loan_id) AS book_on_hand

FROM library_schema."Book" b

JOIN library_schema."Copy" c ON c.book_id = b.book_id

JOIN library_schema."Loan" l ON l.copy_id = c.copy_id

AND l.return_date IS NULL

WHERE

(b.title ILIKE '%c#%'

OR b.title ILIKE '%csharp%'

OR b.title ILIKE '%c sharp%')

AND l.due_date <= CURRENT_DATE + INTERVAL '3 days'

GROUP BY

b.book_id, b.title;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	title	book_on_hand
	text	bigint
1	Programming in C# Basi...	1

3 Создать представления для администрации библиотеки
Нужно создать представления, содержащие

– сведения о должниках;

Решил вывести данные о читателе, количество невозвращенных книг и максимальное количество дней, в течении которых он не возвращает какую-то книгу.

Query

Query History

```

1 CREATE OR REPLACE VIEW library_schema.obligor_readers AS
2 SELECT
3     r.reader_id,
4     r.lats_name,
5     r.first_name,
6     r.middle_name,
7     r.phone,
8     r.email,
9     COUNT(*) AS overdue_books,
10    CURRENT_DATE - MIN(l.due_date) AS max_overdue_days
11 FROM library_schema."Reader" r
12 JOIN library_schema."Loan" l ON l.reader_id = r.reader_id
13 WHERE
14     l.return_date IS NULL
15     AND l.due_date < CURRENT_DATE
16 GROUP BY
17     r.reader_id,
18     r.lats_name,
19     r.first_name,
20     r.middle_name,
21     r.phone,
22     r.email
23 HAVING
24     COUNT(*) >= 1
25 ORDER BY
26     overdue_books DESC;
27
28 SELECT * FROM library_schema.obligor_readers;

```

Data Output

Messages

Notifications

+

📄

▼

🗑️

🔄

📥

📡

SQL

	reader_id integer	lats_name text	first_name text	middle_name text	phone text	email text	overdue_books bigint	max_overdue_days integer
1	2	Морозова	Ольга	Сергеевна	+316987654...	olga.moroz@gmail.c...	2	34
2	3	Васильев	Павел	Андреевич	+316554433...	[null]	1	300

– сведения о наиболее популярных книгах (все экземпляры находятся на руках у читателей).

Вывод представления:

	book_id integer	title text	volume integer	original_language text	publication_type text	area text	publication_city text	publication_year date	bbk_code text	isbn text	total_copies bigint	currently_issued bigint
1	1	Преступление и наказание	1	rus	Роман	Классика	Москва	2019-01-01	84.3(2Рос+Рус...	978-5-17-10823...	3	3
2	10	Programming in C# Basics	1	eng	Учебник	Программирован...	New York	2023-01-01	004.4	978-1-123-4567...	1	1
3	6	Harry Potter and the Chamber of Secr...	1	eng	Фэнтези	Современная про...	London	2002-01-01	84.4(4Англ)	978-0-7475-384...	1	1
4	2	Война и мир	1	rus	Роман-эпопея	Классика	Санкт-Петербург	2020-05-15	84.3(2Рос+Рус...	978-5-389-1456...	1	1
5	7	Test Book 1	1	rus	Тест	Тест	Moscow	2020-01-01	00.0	978-0-000-0001-0	1	1
6	8	Test Book 2	1	rus	Тест	Тест	Moscow	2020-01-01	00.0	978-0-000-0002-0	1	1

Решил вывести немного информации о самой книге, сколько всего копий принадлежит библиотеке и сколько сейчас на руках.

Код представления:

```
CREATE OR REPLACE VIEW library_schema.popular_books AS
SELECT
    b.book_id,
    b.title,
    b.volume,
    b.original_language,
    b.publication_type,
    b.area,
    b.publication_city,
    b.publication_year,
    b.bbk_code,
    b.isbn,
    COUNT(c.copy_id) AS total_copies,
    COUNT(l.loan_id) AS currently_issued
FROM library_schema."Book" b
JOIN library_schema."Copy" c
    ON c.book_id = b.book_id
LEFT JOIN library_schema."Loan" l
    ON l.copy_id = c.copy_id
    AND l.return_date IS NULL
LEFT JOIN library_schema."Reader" r
    ON r.reader_id = l.reader_id
GROUP BY
    b.book_id,
    b.title,
    b.volume,
    b.original_language,
    b.publication_type,
    b.area,
    b.publication_city,
    b.publication_year,
    b.bbk_code,
    b.isbn
HAVING
    COUNT(c.copy_id) = COUNT(l.loan_id)
ORDER BY
    total_copies DESC;

SELECT * FROM library_schema.popular_books;
```

4 Составить 3 запроса на модификацию данных

- Добавляем еще одну самую дорогую копию.

Query	Query History
1	INSERT INTO library_schema."Copy"
2	(copy_id, book_id, inventory_number, price, storage_room, storage_shelf, storage_rack, status)
3	SELECT
4	23,
5	book_id,
6	12321442,
7	price,
8	storage_room,
9	storage_shelf,
10	storage_rack,
11	'available'
12	FROM library_schema."Copy"
13	WHERE price = (
14	SELECT MAX(price)
15	FROM library_schema."Copy"
16	);

Data Output	Messages	Notifications
INSERT 0 1		
Query returned successfully in 45 msec.		

- Ставим статус копии как утерянная, так как она есть в акте замены, который составляется при замене утерянного экземпляра.

Query	Query History
1	UPDATE library_schema."Copy"
2	SET
3	status = 'lost'
4	WHERE copy_id IN (
5	SELECT lost_copy_id
6	FROM library_schema."ReplacementAct"
7	WHERE act_id = 4
8	);

Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 43 msec.		

- Удаляем записи о выдаче экземпляра книги, которым более 2 лет и экземпляр есть в акте о замене при утере.

Query	Query History
1	DELETE FROM library_schema."Loan" l
2	WHERE
3	l.return_date IS NULL
4	AND l.copy_id IN (
5	SELECT lost_copy_id
6	FROM library_schema."ReplacementAct" ra
7	WHERE ra.act_id = 4
8	)
9	AND l.issue_date < CURRENT_DATE - INTERVAL '2 years';

Data Output	Messages	Notifications
DELETE 0		
Query returned successfully in 39 msec.		

5 Создать простой индекс

У меня было мало записей, чтобы увидеть хоть какое-то значительное время выполнения запросов, поэтому я решил добавить.

The screenshot shows a database query editor with a query and its execution plan.

Query:

```
1 INSERT INTO library_schema."Loan" (  
2     loan_id, reader_id, copy_id, issue_date, due_date, return_date, fine_amount, issued_by_employee_id  
3 )  
4 SELECT  
5     generate_series(10000, 20000),  
6     5,  
7     7,  
8     '2026-01-01'::date + (random() * 730)::int,  
9     '2026-01-01'::date + (random() * 730 + 14)::int,  
10    NULL,  
11    (random() * 500)::int,  
12    5  
13 ;  
14  
15 EXPLAIN ANALYZE  
16 SELECT * FROM library_schema."Loan" WHERE reader_id = 5;
```

Data Output Messages Explain x Notifications

QUERY PLAN
text

1	Seq Scan on "Loan" (cost=0.00..86.95 rows=5 width=32) (actual time=0.007..0.540 rows=10002.00 loops=1)
2	Filter: (reader_id = 5)
3	Rows Removed by Filter: 13
4	Buffers: shared hit=74
5	Planning:
6	Buffers: shared hit=2
7	Planning Time: 0.060 ms
8	Execution Time: 0.769 ms

Тут я понял, что только для читателя с айди 5 делать много выдач не правильно. Решил добавить еще случайных записей на выдачу для разных читателей.

```
INSERT INTO library_schema."Loan" (  
    loan_id, reader_id, copy_id, issue_date, due_date, return_date,  
)  
SELECT  
    generate_series(20003, 30000),  
    (random() * 5 + 1)::int,  
    7,  
    '2026-01-01'::date + (random() * 730)::int,  
    '2026-01-01'::date + (random() * 730 + 14)::int,  
    NULL,  
    (random() * 500)::int,  
    5  
;
```

После этого я понял, что индексация для 5 читателя мне не даст интересное время, так как где-то половина записей с этим читателем. Выбрал просто следующего, т.е. 6 читателя.

Результаты для выборки всех выдач книг читателю с идентификатором 6.

Query	Query History
1	EXPLAIN ANALYZE
2	SELECT * FROM library_schema."Loan" WHERE reader_id = 6;
Data Output	Messages Explain × Notifications
	QUERY PLAN text
1	Seq Scan on "Loan" (cost=0.00..471.16 rows=955 width=32) (actual time=0.605..1.221 rows=955.00 loops=1)
2	Filter: (reader_id = 6)
3	Rows Removed by Filter: 19058
4	Buffers: shared hit=221
5	Planning Time: 0.083 ms
6	Execution Time: 1.279 ms

Создаем простой индекс на столбец идентификатора читателя и повторяем запрос.

Query	Query History
1	CREATE INDEX idx_loan_reader_id
2	ON library_schema."Loan" (reader_id);
3	
4	EXPLAIN ANALYZE
5	SELECT * FROM library_schema."Loan" WHERE reader_id = 6;
Data Output	Messages Explain × Notifications
	QUERY PLAN text
1	Bitmap Heap Scan on "Loan" (cost=11.69..244.63 rows=955 width=32) (actual time=0.043..0.147 rows=955.00 loops=1)
2	Recheck Cond: (reader_id = 6)
3	Heap Blocks: exact=74
4	Buffers: shared hit=74 read=2
5	-> Bitmap Index Scan on idx_loan_reader_id (cost=0.00..11.45 rows=955 width=0) (actual time=0.031..0.031 rows=955.00 loops=1)
6	Index Cond: (reader_id = 6)
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=4 read=1
11	Planning Time: 0.871 ms
12	Execution Time: 0.174 ms

Можно заметить, что время исполнения изменилось на порядок. Индекс я вручную удалил после того, как еще несколько раз повторил этот запрос, чтобы убедиться, что это систематическое поведение.

6 Создать составной индекс

Воспользуемся той же таблицей с выдачей книг читателям. Возьмем снова 6 читателя, но будем смотреть на те записи, которые он не вернул (это почти все его записи), а также упорядочим их по дате выдачи.

Query	Query History
1	EXPLAIN ANALYZE
2	SELECT loan_id, copy_id, issue_date, due_date
3	FROM library_schema."Loan"
4	WHERE reader_id = 6
5	AND return_date IS NULL
6	ORDER BY issue_date DESC;

Data Output	Messages	Explain	×	Notifications
QUERY PLAN				
text				
1	Sort (cost=518.43..520.82 rows=955 width=16) (actual time=1.330..1.355 rows=955.00 loops=1)			
2	Sort Key: issue_date DESC			
3	Sort Method: quicksort Memory: 54kB			
4	Buffers: shared hit=221			
5	-> Seq Scan on "Loan" (cost=0.00..471.16 rows=955 width=16) (actual time=0.754..1.208 rows=955.00 loops=1)			
6	Filter: ((return_date IS NULL) AND (reader_id = 6))			
7	Rows Removed by Filter: 19058			
8	Buffers: shared hit=221			
9	Planning Time: 0.071 ms			
10	Execution Time: 1.390 ms			

Добавим составной индекс на идентификатор читателя, на дату возврата и на дату выдачи, для которой укажем убывающий порядок, как и в запросе.

Query	Query History
1	CREATE INDEX idx_loan_reader_return_date
2	ON library_schema."Loan" (reader_id, return_date, issue_date DESC);
3	
4	EXPLAIN ANALYZE
5	SELECT loan_id, copy_id, issue_date, due_date
6	FROM library_schema."Loan"
7	WHERE reader_id = 6
8	AND return_date IS NULL
9	ORDER BY issue_date DESC;

Data Output	Messages	Explain	×	Notifications
QUERY PLAN				
text				
1	Sort (cost=298.28..300.67 rows=955 width=16) (actual time=0.305..0.330 rows=955.00 loops=1)			
2	Sort Key: issue_date DESC			
3	Sort Method: quicksort Memory: 54kB			
4	Buffers: shared hit=74 read=4			
5	-> Bitmap Heap Scan on "Loan" (cost=18.08..251.01 rows=955 width=16) (actual time=0.069..0.186 rows=955.00 loops=1)			
6	Recheck Cond: ((reader_id = 6) AND (return_date IS NULL))			
7	Heap Blocks: exact=74			
8	Buffers: shared hit=74 read=4			
9	-> Bitmap Index Scan on idx_loan_reader_return_date (cost=0.00..17.84 rows=955 width=0) (actual time=0.054..0.054 rows=955.00 loops=1)			
10	Index Cond: ((reader_id = 6) AND (return_date IS NULL))			
11	Index Searches: 1			
12	Buffers: shared read=4			
13	Planning:			
14	Buffers: shared hit=4 read=1			
15	Planning Time: 0.809 ms			
16	Execution Time: 0.365 ms			

Время исполнения улучшилось примерно в 4 раза.

Выводы: в ходе выполнения лабораторной работы были отработаны навыки запроса данных для получения информации из связанных таблиц, создания представлений, запросы на модификацию данных с подзапросами, а также анализ запроса с помощью EXPLAIN ANALYZE и создание индексов после анализа, которые при использовании давали ускорение запросов как минимум в 4 раза.